DS 642: Applications of Parallel Computing

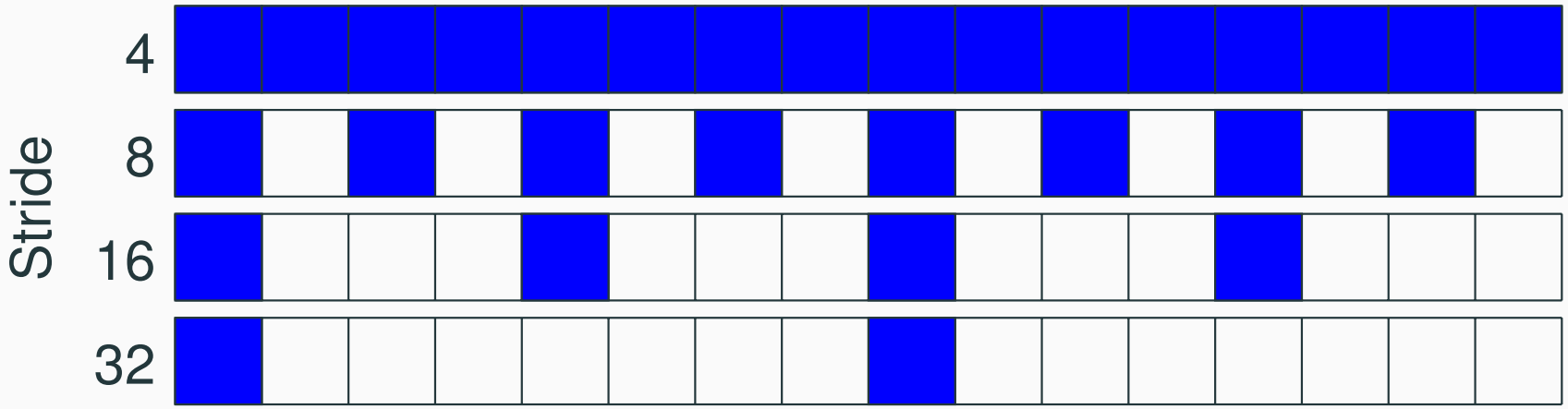
Shahriar Afkhami

Week 2:
Parallelism within single processor
Memory Hierarchies and Matrix-Vector Multiplication

Outline

- Memory benchmark
- Parallelism within single processors
  - SIMD: Single Instruction, Multiple Data
  - FMA: Fused Multiply Add
  - Compiler Optimizations
- Case study: Matrix Multiplication
  - Understand performance optimization
  - Tiled (cache-aware) matrix multiplication

Experimental study of memory: memory benchmark (membench)



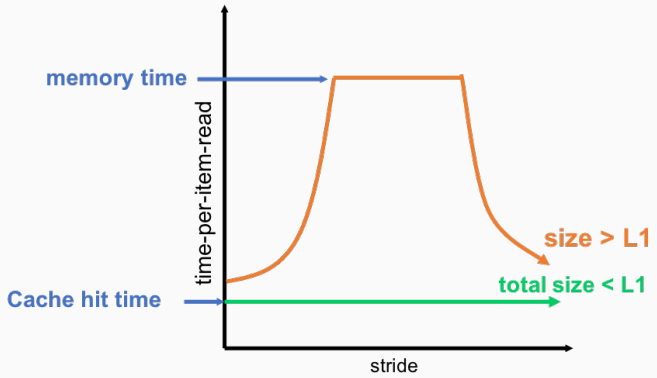
- Size = 64 bytes (16 ints)
- Strides of 4 bytes, 8 bytes, 16 bytes, 32 bytes

```
for array A of length L from 4 KB to 8MB by 2x
  for stride s from 4 bytes to L/2 by 2x
  time the following loop (repeat and average)
    for i = 0 to L by s
      load A[i] from memory (4 bytes)
```

Membench: What to Expect

Consider the average cost per load

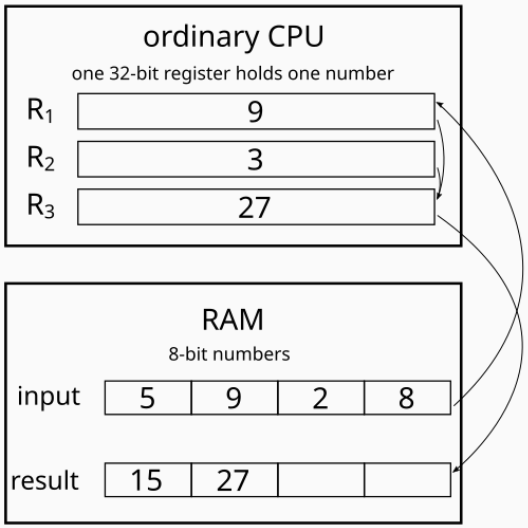
- Plot one line for each array length, time vs. stride
- Small stride is best: if cache line holds 4 words, at most 1/4 miss
- If array is smaller than a given cache, all those accesses will hit (after the first run, which is negligible for large enough runs)
- Picture assumes only one level of cache:



Average cost per access

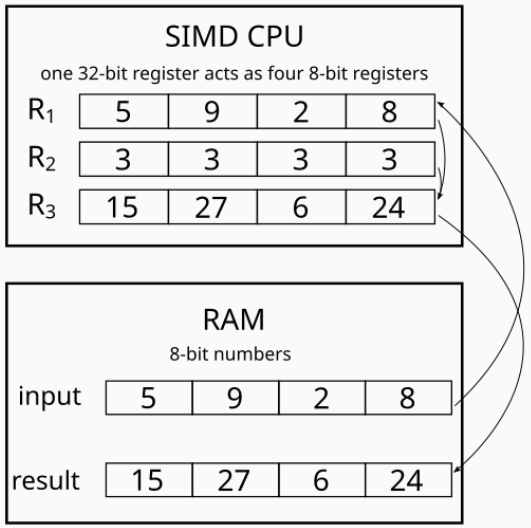
SIMD: *S*ingle *I*nstruction *M*ultiple *D*ata

Conventional uniprocessor: one functional unit per instruction stream



Operation count:
4 loads, 4 multiplies, and 4 saves

SIMD machine: many functional unit per instruction stream



Operation count:
1 load, 1 multiply, and 1 save

FMA: Fused Multiply Add

- Multiply followed by add is very common in programs:

```
x = y + c * z
```

- Executing the multiply/add as a single instruction is at the same rate (bandwidth) as $+$ or $*$ alone.
- FMA is more accurate as it does with a single rounding step.

Compiler Optimizes Code

- Compiler understands how to utilize special features (SIMD, FMA, $\cdots$)
- In practice, needs setting optimization flags:

  `\\no optimization - aggressive optimization -O[0123]`

  - Unrolls/Fuses/Interchanges loops
  - Eliminates dead branches
  - Optimizes instructions
  - Vectorizes when possible

Case study: Matrix Multiplication

- Use simple model of memory to understand performance
- Study communication cost using simple cache model for a warm-up problem: Matrix-vector multiplication
- Naive versus optimized Matrix-Matrix Multiply
  - An important kernel in many problems
    - Dense linear algebra
    - Deep learning training
    - Closely related to many other algorithms
  - Optimization idea can be used in other problems
  - The most studied algorithm in high performance computing

A Simple Model of Memory

- Assume just 2 levels in the hierarchy, fast and slow
- All data initially in slow memory
  - m = number of memory elements (words) moved between fast and slow memory
  - tm = time per slow memory operation (inverse bandwidth in best case)
  - f = number of arithmetic operations
  - tf = time per arithmetic operation $\ll$ tm
  - Computational Intensity: q = f/m, average number of flops per slow memory access
- Minimum possible time = f * tf when all data in fast memory
  - Actual time: f * tf + m * tm = f * tf * (1 + tm/tf * 1/q)
- Larger CI means time closer to minimum f * tf
  - tm/tf is key to machine efficiency
  - q is key to algorithm efficiency
  - q $\geq$ tm/tf needed to get at least half of peak speed

Warm up: Matrix-vector multiplication

Consider a simple serial code:

```
\\read x(1:n) into fast memory
\\read y(1:n) into fast memory
for i = 1:n
  \\read row i of A into fast memory
  for j = 1:n
    y(i) = y(i) + A(i,j)*x(j)
\\write y(1:n) back to slow memory
```

Simplest model:

- m = number of slow memory refs = $3n + n^2$
- f = number of arithmetic operations = $2n^2$
- q =f/m $\approx$ 2

Matrix-vector multiplication limited by slow memory speed

Note on matrix storage

0	5	10	15	20
1	6	11	16	21
2	7	12	17	22
3	8	13	18	23
4	9	14	19	24

Column major

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24

Row major

A matrix is a 2-D array of elements, but memory addresses are "1-D".

- By column, or "column major" (Fortran default): A(i,j) at A+i+j*n
- By row, or "row major" (C default): A(i,j) at A+i*n+j

Matrix multiply

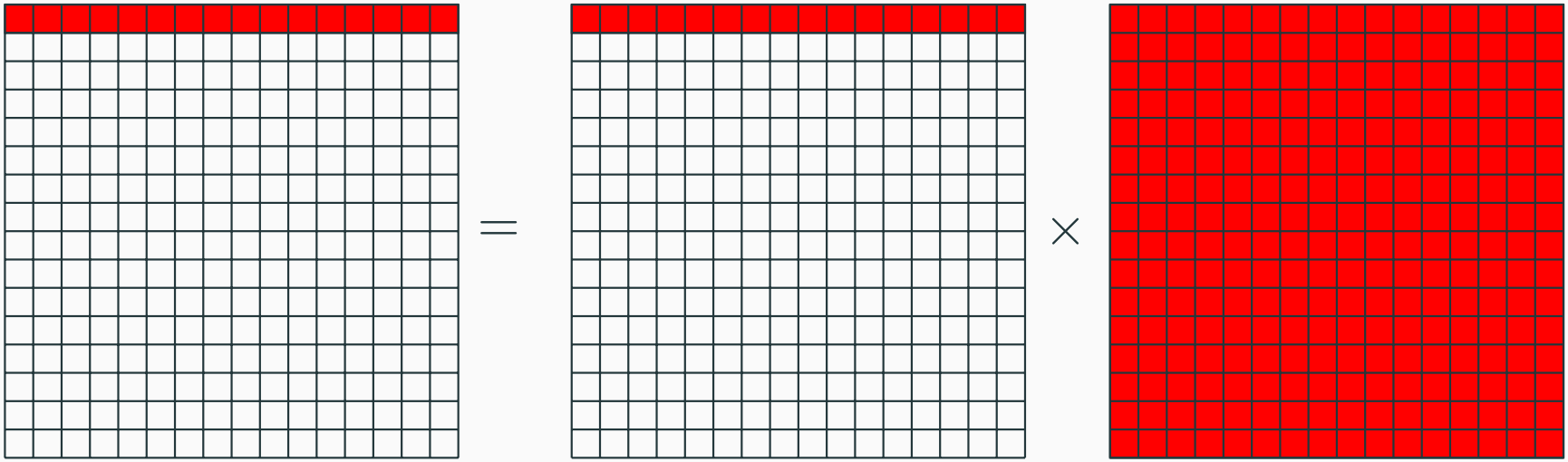
Consider naive square matrix multiplication (column-major order):

```
#define A(i,j) AA[i+j*n]
#define B(i,j) BB[i+j*n]
#define C(i,j) CC[i+j*n]

for (i = 0; i < n; ++i) {
  for (j = 0; j < n; ++j) {
    C(i,j) = 0;
    for (k = 0; k < n; ++k)
      C(i,j) += A(i,k)*B(k,j);
  }
}
```

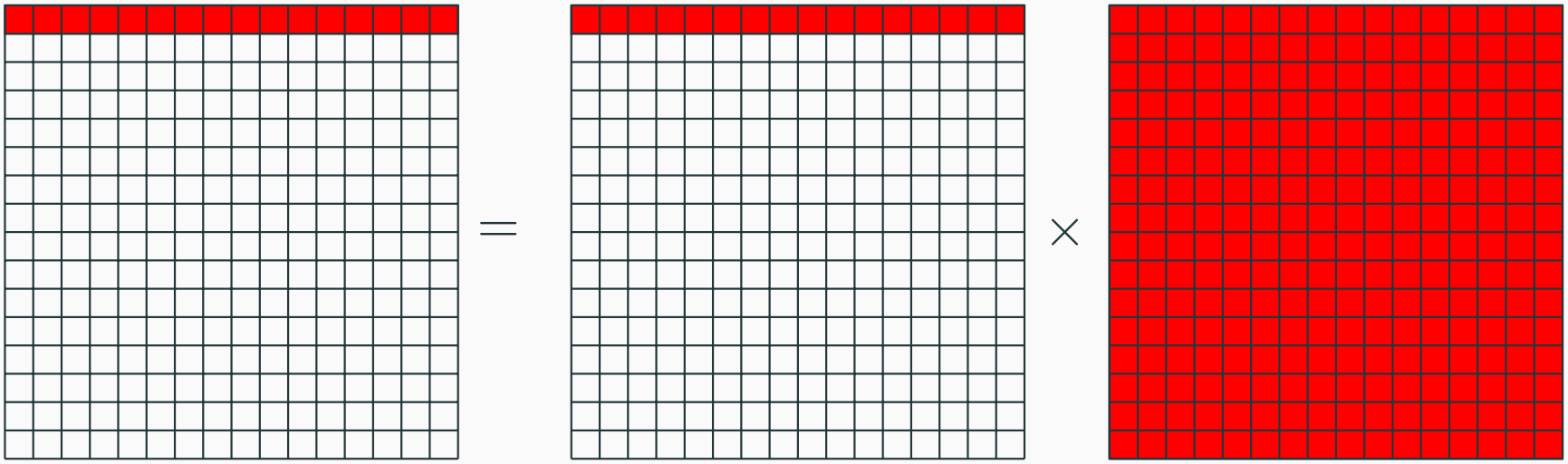

We want to understand the performance of matrix multiply.

Naive Matrix Multiply



- Access A and C with stride of $8M$ bytes
- Access *all* $8M^2$ bytes of B before first re-use
- Poor *arithmetic intensity*

Naive Matrix Multiply



Number of slow memory references on unblocked matrix multiply $m = n^3 + 3n^2$

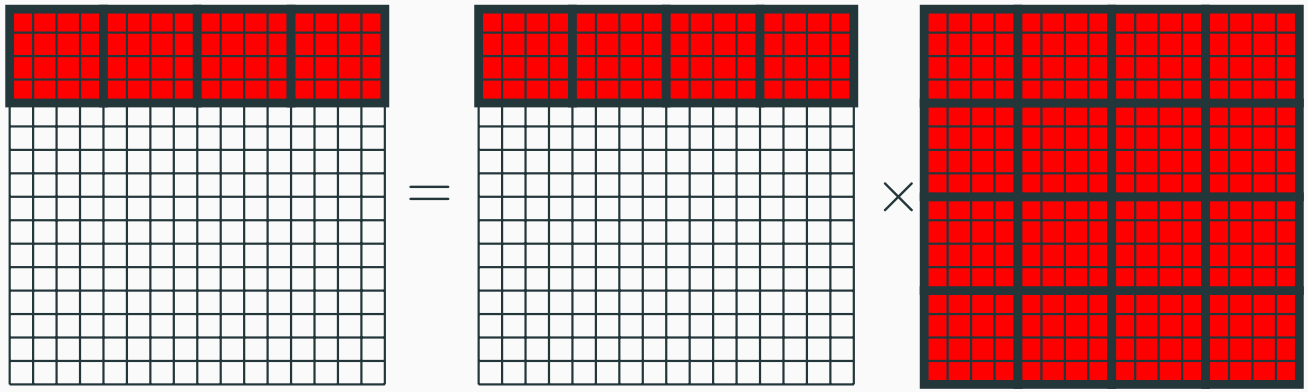
- n^3: to read each column of B n times
- n^2: to read each row of A 1 time
- $2n^2$: to read and write each element of C 1 time

So $q = f/m = 2n^3/(n^3 + 3n^2) \approx 2$ for large n

Blocked (Tiled) Matrix Multiplication

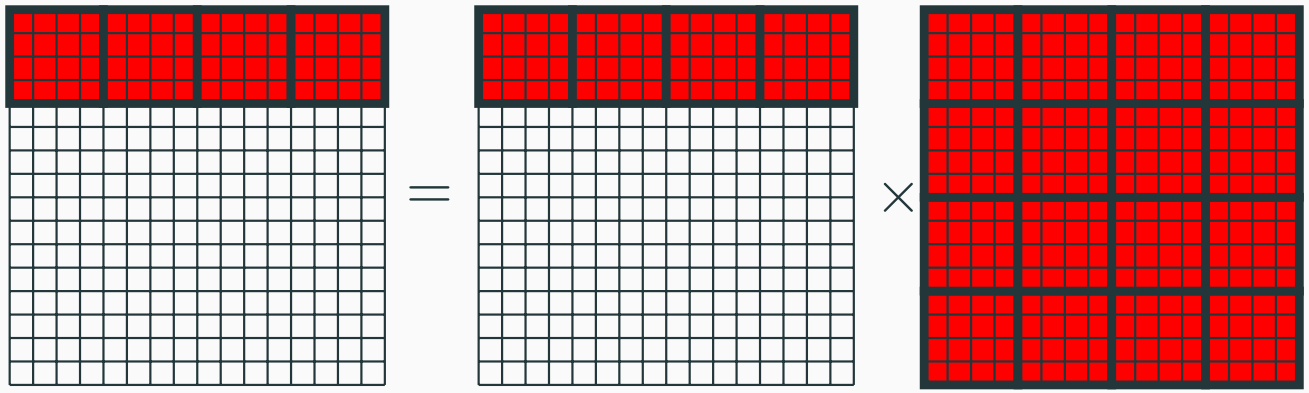
Consider A,B,C to be N-by-N matrices of b-by-b subblocks where $b = n/N$ is called the block size:

```
1  for i = 1:N
2    for j = 1:N
3      \\read block C(i,j) into fast memory
4      for k = 1:N
5        \\read block A(i,k) into fast memory
6        \\read block B(k,j) into fast memory
7        C(i,j) = C(i,j) + A(i,k)*B(k,j) \\do a matrix multiply o
8      \\write block of C(i,j) back to slow memory
```



Blocked Matrix Multiply

The blocked algorithm has computational intensity $q \approx b$



Number of slow memory references on blocked matrix that has $n \times n$ elements, of $N \times N$ blocks each of size $b \times b$ is $m = (2N + 2) * n^2$:

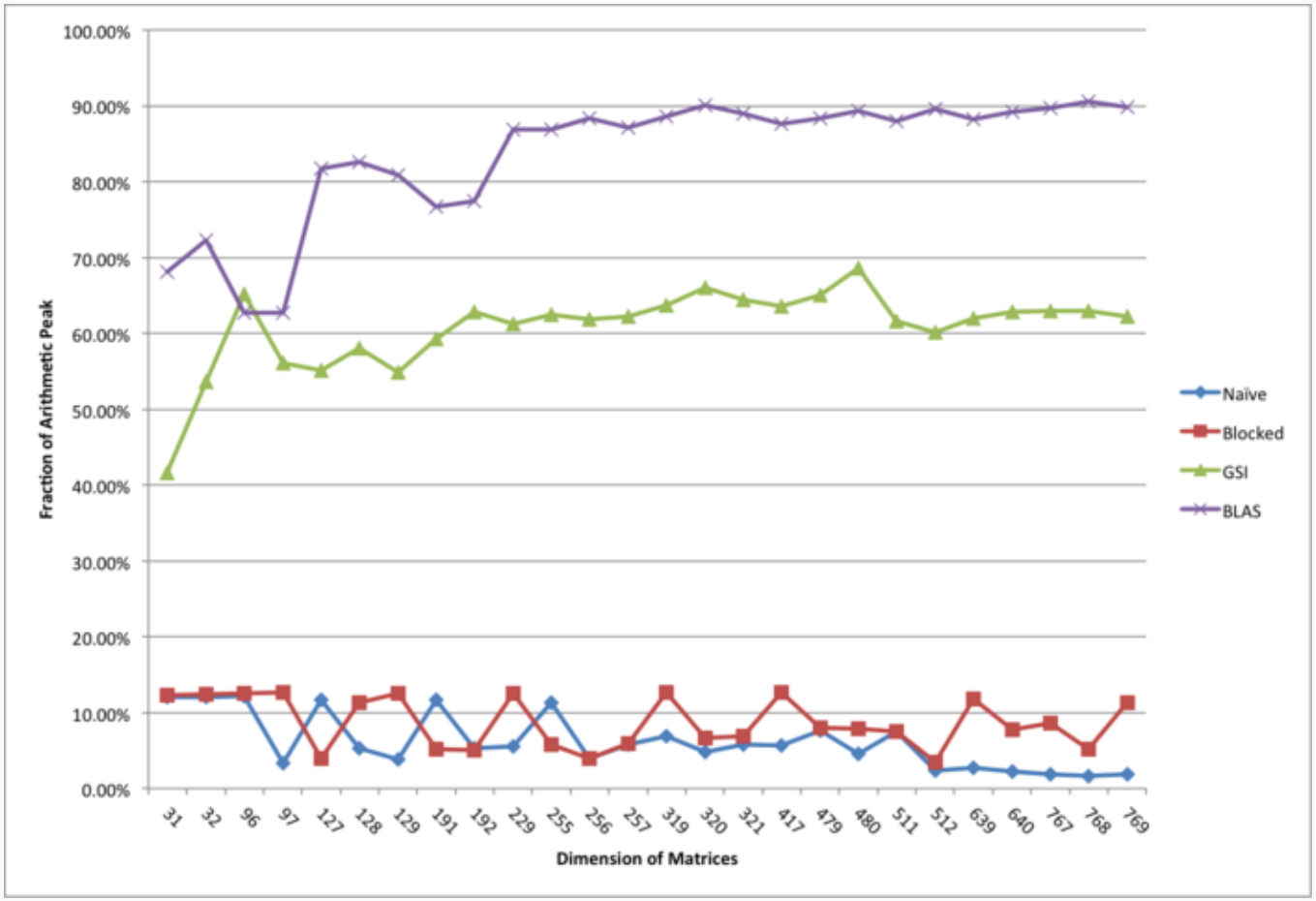
- $N * n^2$: read each block of B N^3 times
- $N * n^2$: read each block of A N^3 time
- $2n^2$: to read and write each block of C 1 time

So $q = f/m = 2n^3/((2N + 2) * n^2) \approx n/N = b$ for large n.

Blocked Matrix Multiply

- The blocked algorithm has computational intensity $q \approx b$.
- So we can improve performance by increasing the blocksize b.
- The larger the block size, the more efficient our algorithm will be.
- All three blocks from A,B,C must fit in fast memory (cache), so we cannot make these blocks arbitrarily large.

Matrix Multiplication: Naive vs Optimized



Timing for matrix multiply; graph from Berkeley CS267 - Lecture 2

Assume your fast memory has size M_f (words), $b \approx \sqrt{M_f/3}$.

Summary of Week 2

- Details of machine are important for performance.
  - Before you parallelize, make sure you're getting good serial performance.
  - Addressing memory is essential.
- There is parallelism hidden within processors.
- Machines have memory hierarchies.
- Locality is at least as important as computation.
  - Temporal: re-use of data recently used
  - Spatial: using data nearby to recently used data
- Can rearrange code/data to improve locality
  - Goal: minimize communication = data movement

HW: You will optimize matrix multiply.